Union-Find

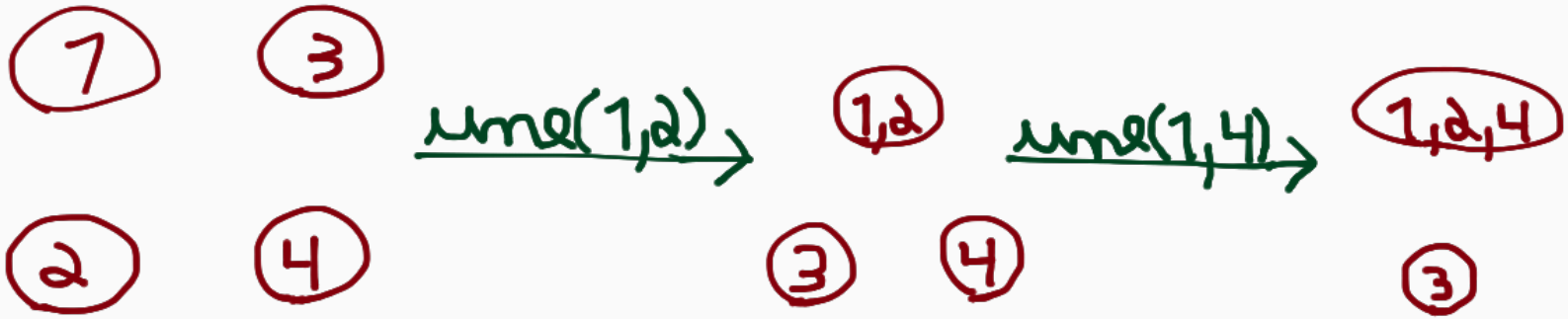
introdução, rollback, persistência parcial, small-to-large

Jonathan Queiroz
https://www.johnjq.com/summer

Union-Find

Apresentação do problema

- Temos vários conjuntos (disjuntos) de elementos
  - Inicialmente, cada conjunto contém precisamente um elemento
- Duas operações:
  - `merge(a, b)`: une os conjuntos aos quais *a* e *b* pertencem
  - `same(a, b)`: verifica se *a* e *b* pertencem ao mesmo conjunto
- Exemplo
  - Esquerda: `same(1, 2) = False`
  - Meio: `same(1, 2) = True`
  - Direita: `same(2, 4) = True`



Abordagem (nem tão) ingênua

Teoria

- Podemos eleger um **líder** para cada conjunto
- Reformulando as operações:
  - `merge(a, b)`: une os conjuntos aos quais *a* e *b* pertencem
  - `find(a)`: retorna o líder do conjunto de *a*
- Podemos reescrever `same(a, b)` como

  `find(a) == find(b)`

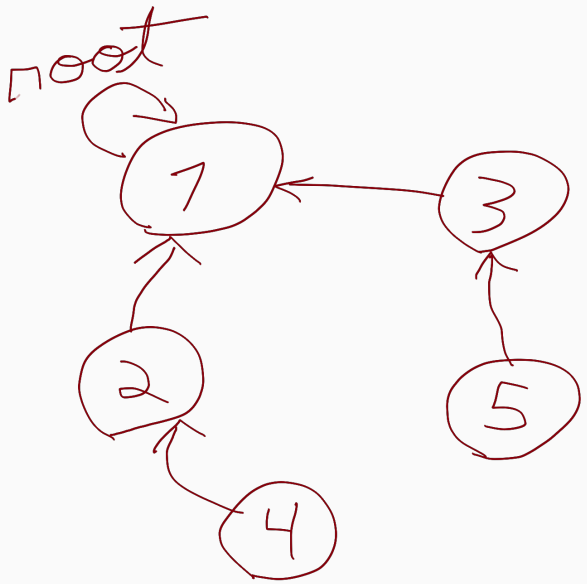


Abordagem (nem tão) ingênua

Ideia da implementação

- Representar cada conjunto como uma árvore
- Basta saber o pai de cada elemento
  - `pai[i]` armazena o índice do pai do i-ésimo elemento
  - `pai[i] = i` indica que o i-ésimo elemento é o líder do seu conjunto

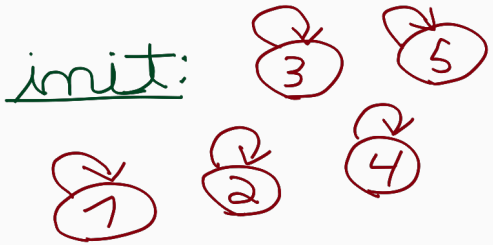
`i`	`pai[i]`
1	1
2	1
3	1
4	2
5	3

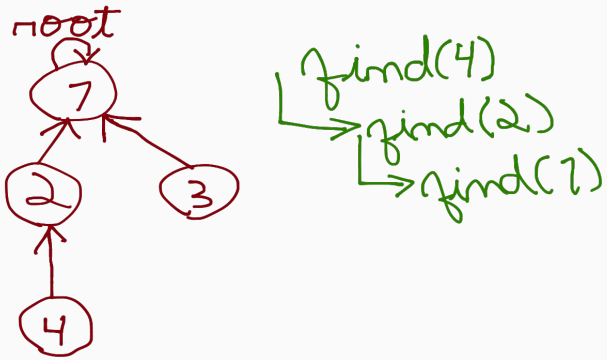


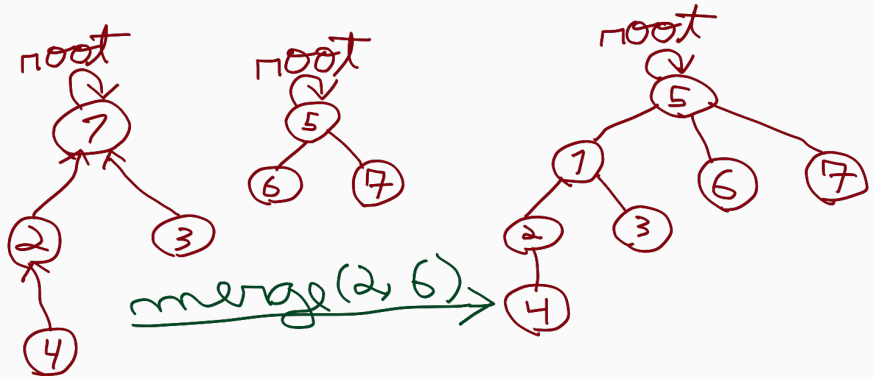
Abordagem (nem tão) ingênua

Implementação

```
int pai[MAX_N+1];
void init() {
    for (int i = 1; i <= MAX_N; ++i)
        pai[i] = i;
}
int find(int x) {
    if (pai[x] == x)
        return x;
    else
        return find(pai[x]);
}
void merge(int u, int v) {
    int a = find(u);
    int b = find(v);
    pai[a] = b;
}
```

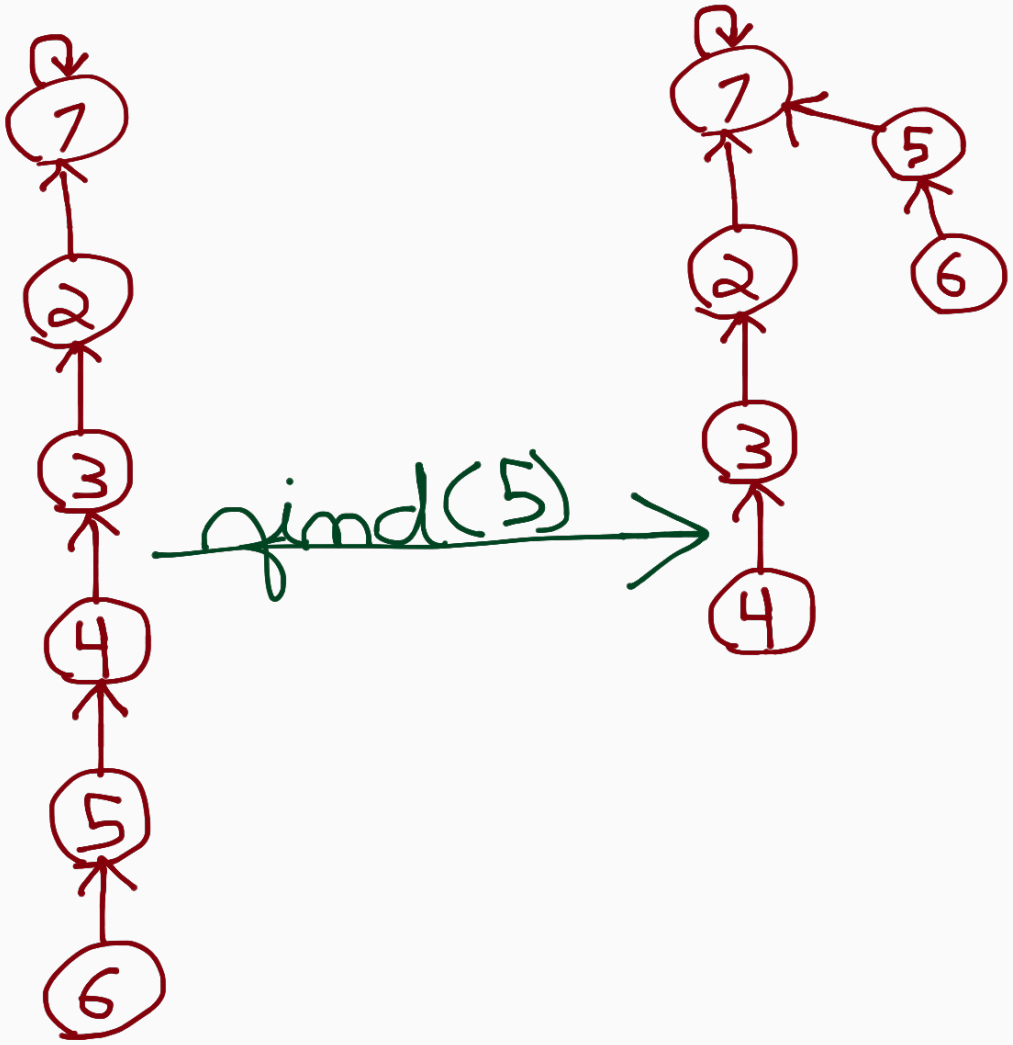






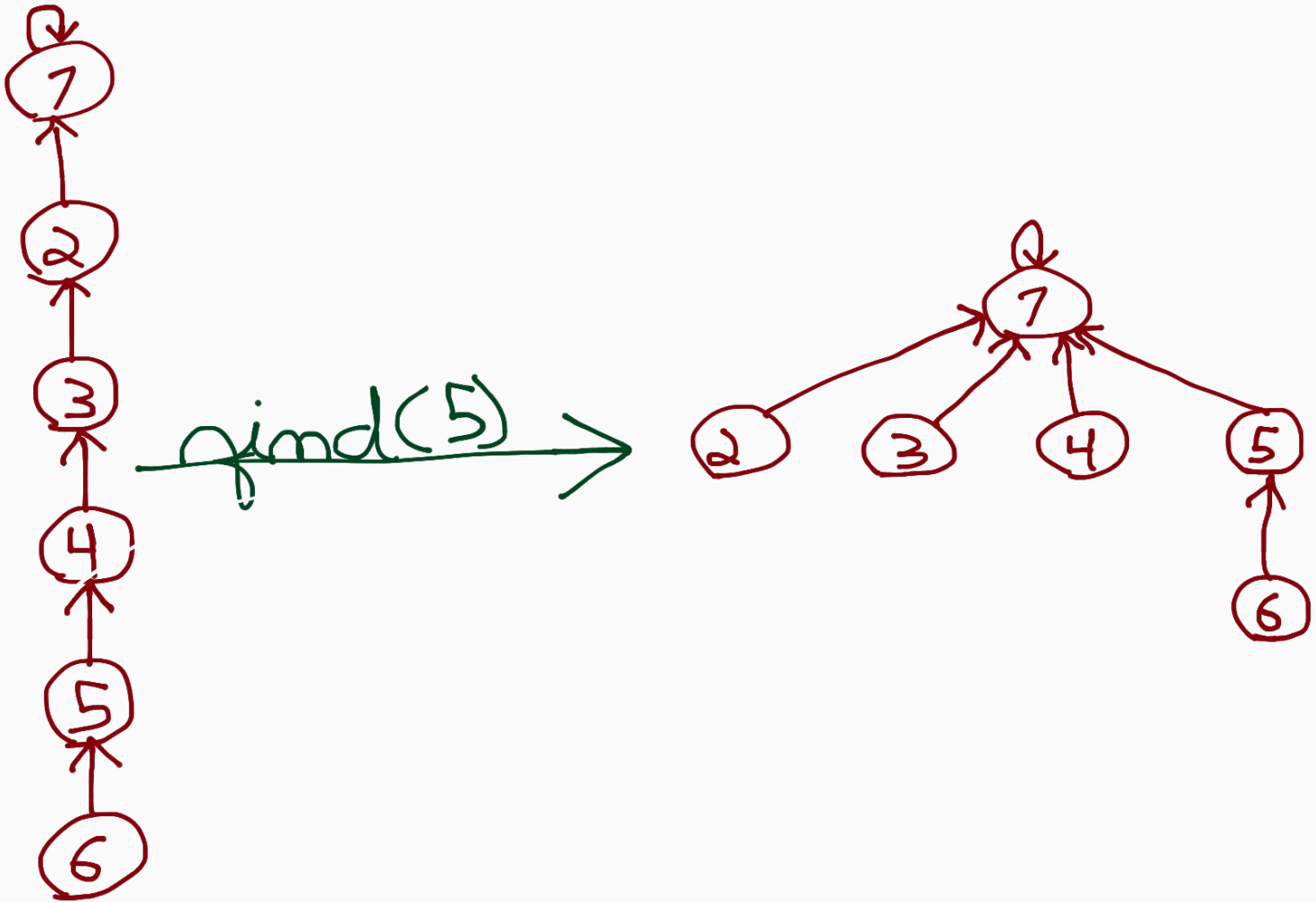
Abordagem (nem tão) ingênua

Como melhorar?



Abordagem (nem tão) ingênua

Como melhorar ainda mais?



Abordagem: path compression

Teoria

- A cada operação `find`: apontar todos os nós acessados diretamente para a raiz
- Intuição: buscas custosas ajudam a melhorar a árvore
- Tempo por operação: $O(\log n)$ *amortizado*
    - Operações individuais podem ter custo elevado
        - Exemplo?
    - Mas o custo médio das operações é baixo
    - Formalmente: m operações levam tempo $O(n + m \log n)$

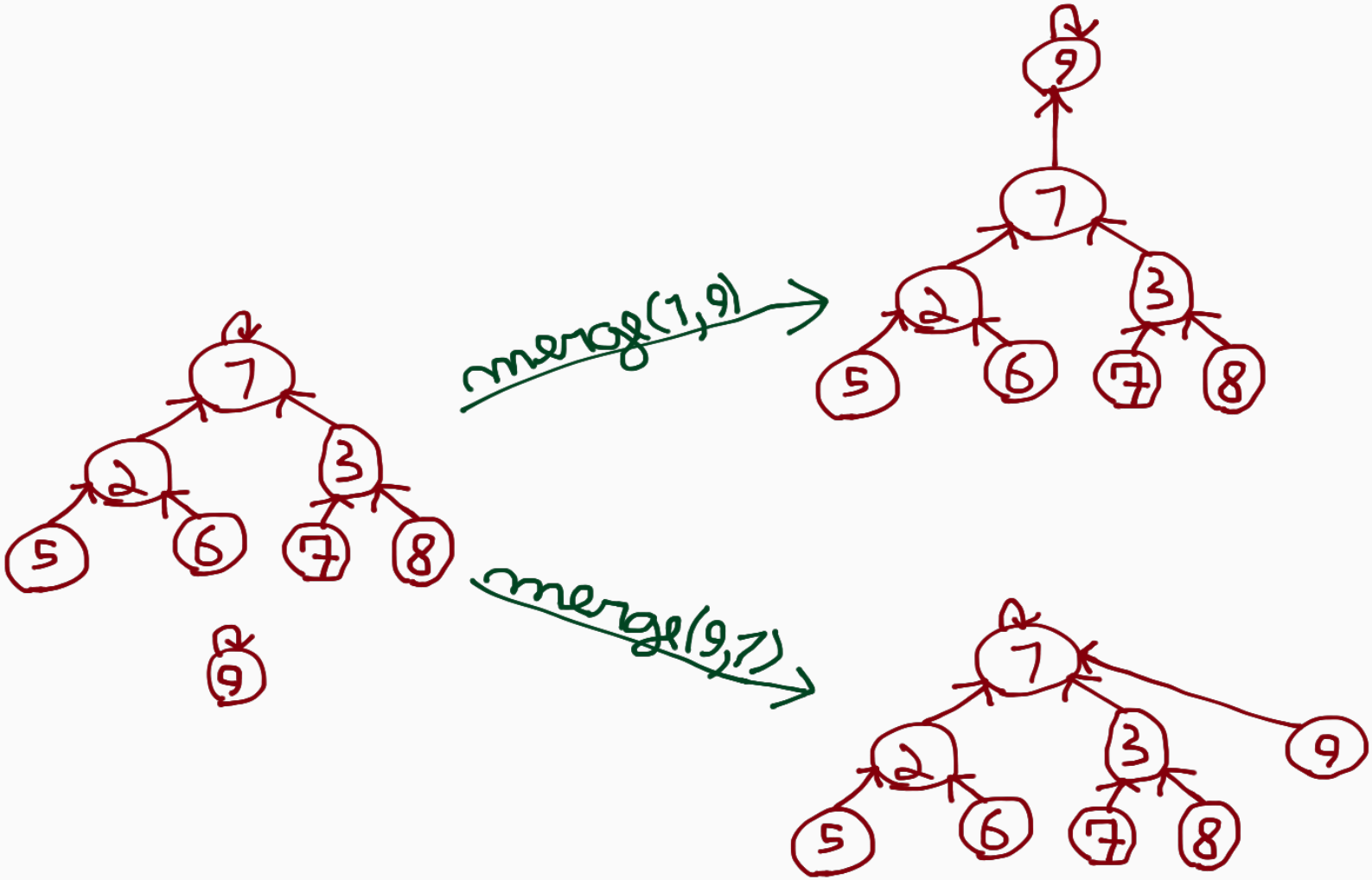
Abordagem: path compression

Implementação

```
int pai[MAX_N+1];
void init() {
    for (int i = 1; i <= MAX_N; ++i)
        pai[i] = i;
}
int find(int x) {
    if (pai[x] == x)
        return x;
    else
        return pai[x] = find(pai[x]);
}
void merge(int u, int v) {
    int a = find(u);
    int b = find(v);
    pai[a] = b;
}
```

Abordagem: union by size

Outra forma de melhorar



Abordagem: union by size

Teoria

- Unir conjuntos do menor para o maior
  - É necessário manter o tamanho de cada conjunto v
- Intuição: minimizar a profundidade dos conjuntos
- Tempo por operação: $O(\log n)$
  - Provaremos em breve

Implementação (parte 1)

```
int pai[MAX_N+1];
int tam[MAX_N+1];
void init() {
    for (int i = 1; i <= MAX_N; ++i) {
        pai[i] = i;
        tam[i] = 1;
    }
}
```

Abordagem: union by size

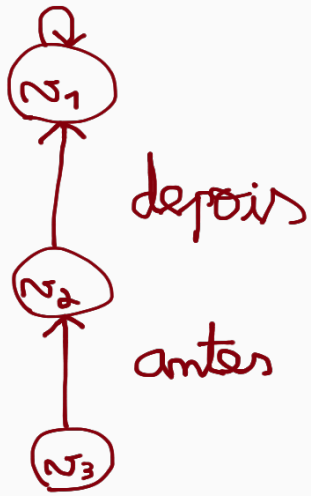
Implementação (parte 2)

```
int find(int x) {
    if (pai[x] == x) return x;
    else             return find(pai[x]);
}
void merge(int u, int v) {
    int a = find(u);
    int b = find(v);
    if (tam[a] < tam[b]) {
        pai[a] = b;
        tam[b] += tam[a];
    } else {
        pai[b] = a;
        tam[a] += tam[b];
    }
}
```

Abordagem: union by size

Análise de complexidade (versão "small-to-large")

- Observação
  - Seja $v_1 v_2 \ldots v_n$ um caminho, sendo v_1 o líder do conjunto
  - Então as uniões foram feitas na seguinte ordem:
    - conjunto de v_n com conjunto de v_{n-1}
    - conjunto de v_{n-1} com conjunto de v_{n-2}
    - ...
    - conjunto de v_3 com conjunto de v_2
    - conjunto de v_2 com conjunto de v_1
  - Por quê?



- Consequência
  - A cada união feita, o tamanho do conjunto pelo menos dobra
  - Logo a profundidade de um nó não pode exceder $\log_2 n$

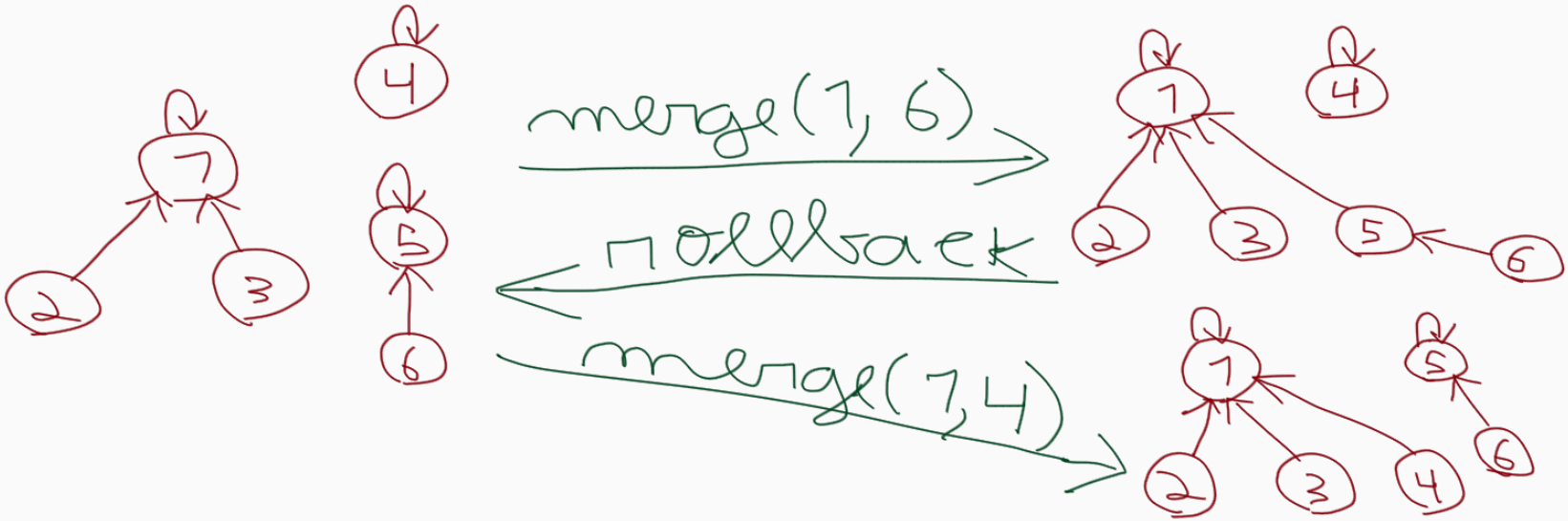
Problema: War (UVa-10158)

- Existem N países
- Inicialmente não há relações entre diferentes países
- Quatro tipos de operações precisam ser suportadas:
  - Estabelecer uma relação de amizade entre x e y (˜)
  - Estabelecer uma relação de inimizade entre x e y (*)
  - Consultar se x e y são amigos
  - Consultar se x e y são inimigos
- Semântica
  - ˜ é uma relação de equivalência
  - * é simétrica e irreflexiva
  - "O inimigo do meu inimigo é meu amigo"
  - "O inimigo do meu amigo é meu amigo"

Rollback

Rollback com union by size

- Podemos desfazer as uniões imediatamente anteriores



- A cada operação `merge`, são feitas duas atribuições:
  - `pai[i] = x;`
  - `tam[j] = y;`
- Basta salvar os valores antigos em uma pilha!

Rollback com union by size

Implementação (parte 1)

```
int pai[MAX_N+1];
int tam[MAX_N+1];
stack<pair<int, int>> old_pai;
stack<pair<int, int>> old_tam;
void init() {
    for (int i = 1; i <= MAX_N; ++i) {
        pai[i] = i;
        tam[i] = 1;
    }
}
int find(int x) {
    if (pai[x] == x) return x;
    else             return find(pai[x]);
}
```

Rollback com union by size

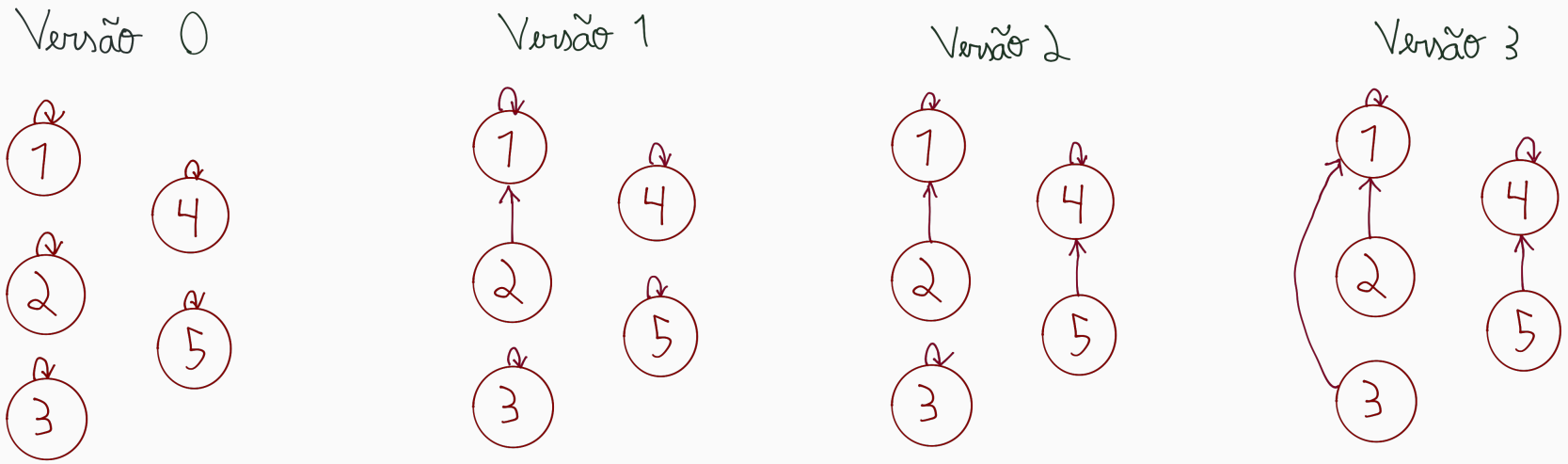
Implementação (parte 2)

```
void merge(int u, int v) {
    int a = find(u);
    int b = find(v);
    if (tam[a] > tam[b]) swap(a, b);
    old_pai.emplace(a, pai[a]);
    old_tam.emplace(b, tam[b]);
    pai[a] = b;
    tam[b] += tam[a];
}
void rollback() {
    pai[old_pai.top().first] = old_pai.top().second;
    tam[old_tam.top().first] = old_tam.top().second;
    old_pai.pop();
    old_tam.pop();
}
```

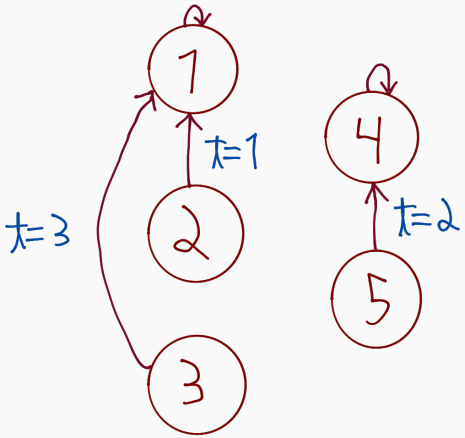
Persistência parcial

Persistência parcial com union-by-size

- Podemos consultar versões anteriores da estrutura

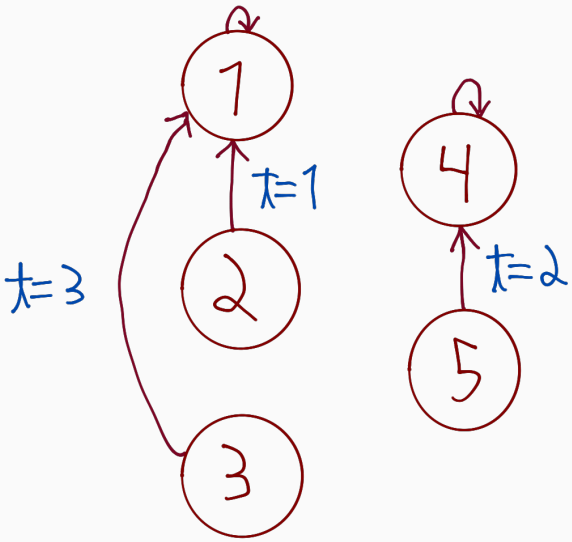


- Ideia: anotar o tempo de cada ligação



Persistência parcial com union-by-size

- Ideia: anotar o tempo de cada ligação



- Operações suportadas:
  - `merge(a, b)`: une os conjuntos aos quais a e b pertencem
  - `find(a, t)`: retorna o líder do conjunto de a no tempo t
  - `same(a, b, t)`: verifica se a e b pertencem ao mesmo conjunto no tempo t

Persistência parcial com union-by-size

Implementação (parte 1)

```
int tempo;
int pai[MAX_N+1];
int tam[MAX_N+1];
int his[MAX_N+1];
void init() {
    tempo = 0;
    for (int i = 1; i <= MAX_N; ++i) {
        pai[i] = i;
        tam[i] = 1;
        his[i] = 0;
    }
}
```

Persistência parcial com union-by-size

Implementação (parte 2)

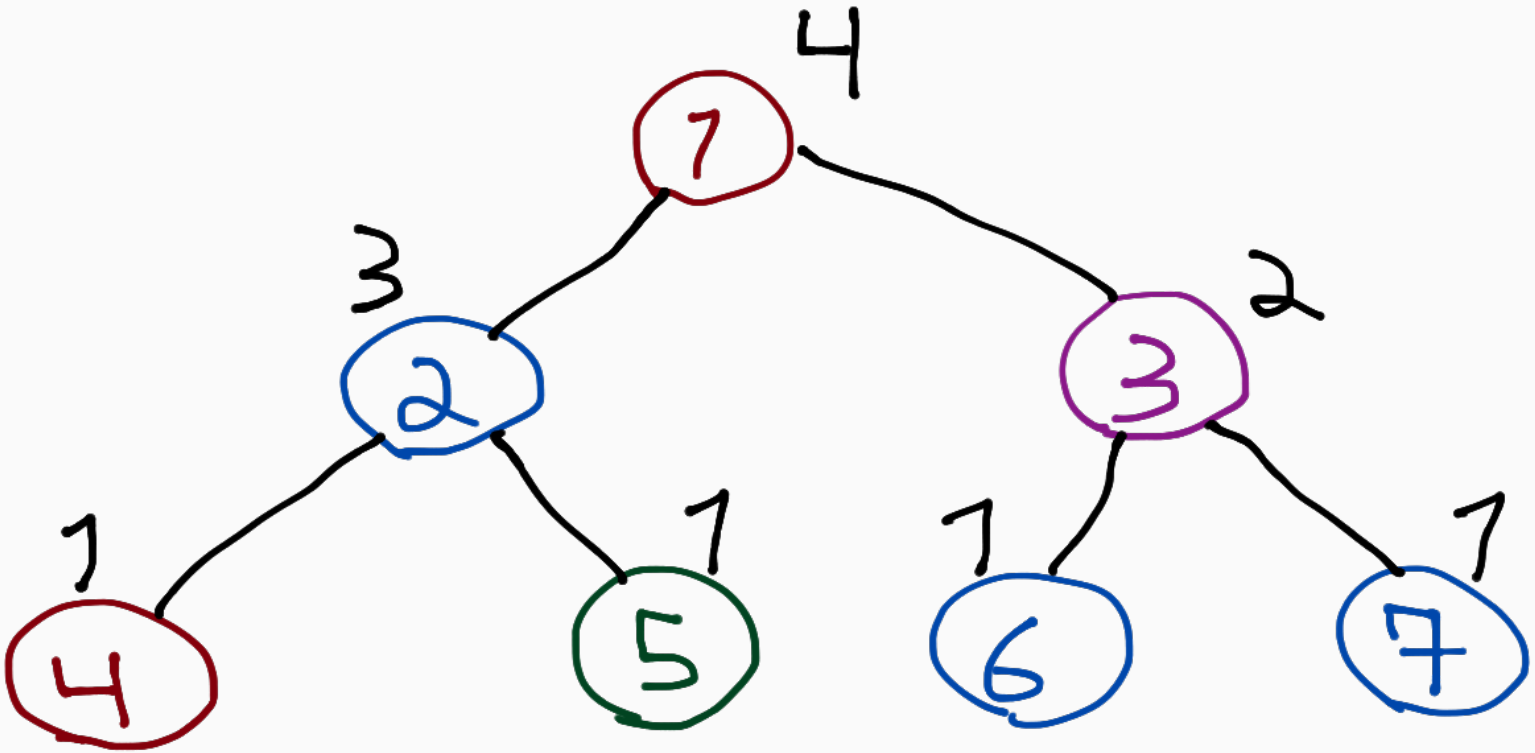
```
int find(int x, int t) {
    if (pai[x] == x) return x;
    if (his[x] > t)  return x;
    return find(pai[x], t);
}
void merge(int u, int v) {
    tempo += 1;
    int a = find(u, tempo);
    int b = find(v, tempo);
    if (tam[a] > tam[b]) swap(a, b);
    pai[a] = b;
    his[a] = tempo;
    tam[b] += tam[a];
}
```

Small-to-large

Cores em subárvores

Descrição do problema

- Entrada: árvore na qual os vértices possuem cores
- Objetivo: encontrar a quantidade de cores da subárvore enraizada em cada vértice



Cores em subárvores

Abordagem ingênua

- Busca em profundidade
- Para cada nó, unir os conjuntos de cores dos seus filhos

```
set<int> dfs(int v) {
    set<int> colors;
    colors.insert(color[v]);
    for (int w : adj[v])
        for (int c : dfs(w))
            colors.insert(c);
    ans[v] = colors.size();
    return colors;
}
```

- Complexidade no pior caso: $\Theta(n^2 \log n)$
  - Como construir tal caso?

Cores em subárvores

Abordagem "small-to-large"

- Mesma ideia, mas unindo conjuntos do menor para o maior

```
set<int> colors[MAX_N+1];
void dfs(int v) {
    colors[v].insert(color[v]);
    for (int w : adj[v]) {
        dfs(w);
        if (colors[w].size() > colors[v].size())
            swap(colors[v], colors[w]);
        for (int c : colors[w])
            colors[v].insert(c);
    }
    ans[v] = colors[v].size();
}
```

- Complexidade no pior caso: $\Theta(n \log^2 n)$
  - Mesmo argumento do *union by size*!

Prática

Exercícios diretos

- Fusões (SPOJ-BR)
- Coleção de livros (codeforces)
    - necessário entrar no grupo da UFBA no codeforces (clique aqui)
- Tree (e-olymp)